California_Housing

Introduzione

Questo documento raccoglie e organizza in modo coerente tutti i concetti teorici, le metodologie e le best practice apprese attraverso l'analisi esplorativa del dataset California Housing. Il materiale copre l'intera pipeline di pre-processing e analisi di un dataset, dall'importazione fino alla normalizzazione, passando per la gestione dei dati mancanti, degli outlier, delle variabili categoriche e la creazione di nuove feature.

1. Analisi Esplorativa Iniziale (EDA)

1.1 Comprendere la Struttura del Dataset

Prima di qualsiasi analisi, è fondamentale comprendere la struttura del dataset. Due strumenti essenziali di pandas per questo scopo sono `describe()` e `info()`.

Il Metodo `describe()`

Il metodo `describe()` fornisce una sintesi statistica delle colonne numeriche:

```
df.describe()
```

Statistiche chiave da analizzare:

- **Count**: verifica che non ci siano valori mancanti (tutti i conteggi devono essere uguali)
- **Mean (media)**: la media aritmetica dei valori
- **Std (deviazione standard)**: misura la dispersione dei dati rispetto alla media
- **Min/Max**: i valori estremi, da verificare per plausibilità
- **Quartili (25%, 50%, 75%)**: permettono di comprendere la distribuzione

Cosa controllare nel output di `describe()`:

1. **Plausibilità di min e max**: valori fuori scala indicano potenziali errori o outlier (es. `AveRooms` con max 141.9 è sospetto)
2. **Rapporto media/deviazione standard**: se la deviazione standard è maggiore della media,

indica forte dispersione dei dati
3. **Simmetria della distribuzione**: confronta media e mediana (50%). Se differiscono significativamente, la distribuzione è **skewed** (asimmetrica). Se la media è inferiore alla mediana, la distribuzione ha skew negativo (coda verso sinistra)

Il Metodo `info()`

Il metodo `info()` fornisce informazioni sulla struttura del DataFrame:

```
df.info()
```

Cosa controllare nel output di `info()`:

1. **Colonne nulle**: verifica quali colonne hanno valori mancanti (`Non-Null Count < count totale`)
2. **Tipi di dato**: conferma che le colonne siano del tipo appropriato (numeri come float64/int64, categoriche come object)
3. **Utilizzo memoria**: utile per dataset grandi

1.2 Gestione dei Dati Mancanti (Missing Values)

I dati mancanti sono un problema comune nell'analisi reale. La strategia di gestione dipende dalla percentuale di valori mancanti:

Strategie di Gestione

Percentuale Null	Strategia Consigliata
< 5%	Rimuovi le righe con valori nulli
5% - 20%	Imputa con media (numerici) o moda (categoriche)
> 20%	Valuta di eliminare la colonna o usare un modello ML per prevedere i valori mancanti

Imputazione Avanzata con Gruppi

Invece di usare la media globale, un approccio più sofisticato consiste nell'imputare usando la media **condizionata** a una variabile correlata.

Esempio: se hai il 10% di valori nulli sul guadagno, raggruppa per fascia d'età e sostituisci i null

con la media dei guadagni **per quella fascia**. Questo rispecchia meglio la realtà perché tiene conto della relazione tra età e guadagno.

```
# Esempio concettuale
df['Guadagno'] = df.groupby('Fascia_Età')['Guadagno'].transform(
    lambda x: x.fillna(x.mean())
)
```

1.3 Identificazione e Gestione degli Outlier

Un **outlier** è un'osservazione che si discosta significativamente dal comportamento generale del dataset. Gli outlier possono essere:

- **Statistici**: rari ma plausibili (es. persona alta 2.30 metri)
- **Errori**: chiaramente errati (es. persona alta 23 metri)

Perché gli Outlier Sono Importanti

1. **Qualità del dato**: possono indicare errori di misurazione o inserimento
2. **Comprensione del fenomeno**: possono rappresentare eventi rari ma significativi (es. case di lusso)
3. **Impatto sui modelli**: molti algoritmi (regressione lineare, media, varianza) sono sensibili ai valori estremi

Il Boxplot e l'IQR (Interquartile Range)

Il **boxplot** è uno strumento grafico che visualizza la distribuzione di una variabile numerica mostrando:

- **Mediana**: linea centrale nella scatola
- **Q1 e Q3**: primo e terzo quartile (delimitano il 50% centrale dei dati)
- **IQR**: intervallo interquartile, calcolato come `Q3 - Q1`
- **Baffi**: estendono fino a `Q1 - 1.5 × IQR` e `Q3 + 1.5 × IQR`
- **Outlier**: punti isolati oltre i baffi

Formula per identificare outlier con IQR:

```
lower_bound = Q1 - 1.5 × IQR
upper_bound = Q3 + 1.5 × IQR
outlier = valore < lower_bound OR valore > upper_bound
```

Implementazione:

```
def remove_outliers_iqr(data, columns):
    df_clean = data.copy()
    for col in columns:
        Q1 = df_clean[col].quantile(0.25)
        Q3 = df_clean[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        df_clean = df_clean[(df_clean[col] >= lower_bound) &
                            (df_clean[col] <= upper_bound)]
    return df_clean
```

Il Metodo Z-Score

Lo **Z-score** misura quanti deviazioni standard un valore si trova dalla media:

$$Z = \frac{x - \mu}{\sigma}$$

Dove:

- x è il valore
- μ è la media
- σ è la deviazione standard

Un valore con $|Z| > 3$ è considerato un outlier estremo (meno dello 0.3% dei dati in una distribuzione normale).

Implementazione:

```
from scipy import stats
import numpy as np

def remove_outliers_zscore(data, columns, threshold=3):
    df_clean = data.copy()
    z_scores = np.abs(stats.zscore(df_clean[columns]))
    filtered_entries = (z_scores < threshold).all(axis=1)
    return df_clean[filtered_entries]
```

Confronto tra IQR e Z-Score

Caratteristica	IQR	Z-Score

Caratteristica	IQR	Z-Score
Robustezza	Alta (non assume distribuzione)	Bassa (assume normalità)
Aggressività	Più aggressivo nella rimozione	Meno aggressivo
Quando usare	Distribuzioni sporche o sconosciute	Distribuzioni normali o vicine

1.4 Distribuzioni Incrociate e Pairplot

Il **pairplot** è uno strumento potente per visualizzare le relazioni tra tutte le coppie di variabili in un dataset:

```
import seaborn as sns

df_sample = df.sample(n=1000, random_state=42)
sns.pairplot(df_sample, diag_kind='kde', plot_kws={'alpha': 0.4, 's': 10})
plt.show()
```

Come interpretare un pairplot:

- **Diagonale**: mostra la distribuzione di ogni variabile (istogrammi o KDE)
  - Forme a campana = distribuzione normale
  - Forme asimmetriche = distribuzione skewed
  - Picchi multipli = distribuzione multimodale
- **Fuori diagonale**: mostra scatter plot di tutte le coppie di variabili
  - Pattern lineare = correlazione lineare
  - Pattern curvilineo = correlazione non lineare
  - Nuvola senza struttura = nessuna correlazione

Cosa cercare:

1. **Linearità**: forme a "cannone" o "sigaro" indicano dipendenza lineare
2. **Outlier**: punti isolati negli scatter plot
3. **Cluster**: gruppi di punti che suggeriscono sottopopolazioni

1.5 Matrici di Correlazione

La **correlazione** misura la relazione lineare tra due variabili. Il coefficiente di correlazione di

Pearson varia da -1 a +1:

- **+1**: correlazione positiva perfetta (se una sale, l'altra sale proporzionalmente)
- **0**: nessuna correlazione lineare
- **-1**: correlazione negativa perfetta (se una sale, l'altra scende proporzionalmente)

Interpretazione dei Valori

Valore	Interpretazione
0.0 - 0.15	Nessuna o molto debole
0.15 - 0.3	Debole
0.3 - 0.5	Moderata
0.5 - 0.7	Forte
0.7 - 0.9	Molto forte
0.9 - 1.0	Estremamente forte

Nel contesto del dataset California Housing, si considerano interessanti le correlazioni > |0.15|.

ATTENZIONE: correlazione non implica causalità. Due variabili possono essere correlate senza che una causi l'altra (es. entrambi influenzati da una terza variabile nascosta).

Correlazione di Pearson

La correlazione di Pearson è la misura standard per relazioni lineari:

```
corr = df.corr(method='pearson')  # default
```

Correlazione di Spearman

Quando le relazioni sono **non lineari** o ci sono molti outlier, la correlazione di Spearman è più appropriata. Spearman misura la correlazione **monotona** basandosi sui ranghi invece che sui valori originali:

```
corr_spearman = df.corr(method='spearman')
```

Quando usare Spearman invece di Pearson:

- Dati con outlier estremi
- Relazioni non lineari ma monotone

- Dati ordinali
- Distribuzioni non normali

Visualizzazione con Heatmap

```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Crea maschera per metà superiore (la matrice è simmetrica)
mask = np.triu(np.ones_like(corr, dtype=bool))

plt.figure(figsize=(12, 10))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f',
            cmap='coolwarm', vmin=-1, vmax=1, center=0)
plt.title('Matrice di Correlazione')
plt.show()
```

2. Feature Engineering

Il **feature engineering** è il processo di creazione di nuove feature (variabili) a partire da quelle esistenti, con l'obiettivo di migliorare le prestazioni dei modelli di machine learning o di comprendere meglio i dati.

2.1 Tecniche di Feature Engineering

Raggruppamento di Variabili Continue (Binning)

Trasforma una variabile continua in una variabile categorica o ordinata raggruppando i valori in intervalli:

```
# Raggruppa HouseAge in fasce di 5 anni
df['HouseAge_Group'] = pd.cut(df['HouseAge'],
                              bins=range(0, 56, 5),
                              labels=range(0, 55, 5)).astype(float)
```

Usa binning quando:

- Vuoi identificare soglie o punti di svolta
- La relazione con il target non è lineare

- Vuoi rendere le feature più interpretabili

Creazione di Classi con Quartili

Usa `qcut` per creare classi con un numero uguale di osservazioni:

```
# Crea 3 classi di reddito (terzili)
df['Income_Class'] = pd.qcut(df['MedInc'], q=3, labels=[1, 2, 3])
```

Differenza tra `cut` e `qcut`:

- `cut` : divide in intervalli di uguale **ampiezza**
- `qcut` : divide in intervalli con uguale **numero di osservazioni**

Creazione di Feature Derivate

Combina feature esistenti attraverso operazioni matematiche:

```
# Rapporto tra due feature
df['Occup_per_Room'] = df['AveOccup'] / df['AveRooms']

# Potenza di una feature (per relazioni non lineari)
df['AveRooms_Squared'] = df['AveRooms'] ** 2
```

Esempi pratici:

- **Affollamento per stanza**: `AveOccup / AveRooms` indica qualità abitativa
- **Indice di Massa Corporea**: `peso / altezza^2` in ambito medico
- **Debito/Rent**: rapporto finanziario in ambito creditizio

Feature da Conoscenza del Dominio

Crea feature basate su conoscenza specifica del settore:

```
# Distanza da centri economici (usando formula Haversine)
def haversine_km(lat1, lon1, lat2, lon2):
    R = 6371  # raggio terrestre in km
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
    c = 2 * np.arcsin(np.sqrt(a))
    return R * c

df['Dist_SF_km'] = haversine_km(df['Latitude'], df['Longitude'], 37.7749, -122.4194)
df['Dist_LA_km'] = haversine_km(df['Latitude'], df['Longitude'], 34.0522, -118.2437)
df['Dist_min_km'] = df[['Dist_SF_km', 'Dist_LA_km']].min(axis=1)
```

2.2 Interpretazione dei Risultati di Feature Engineering

Dall'analisi del dataset California Housing sono emersi diversi pattern importanti:

Pattern 1: Relazione Reddito-Prezzo

Il prezzo medio delle case cresce in modo **quasi lineare** lungo i primi 8 decili di reddito. Nell'ultimo decile si osserva un'accelerazione marcata, segno che le aree con reddito più elevato presentano un salto strutturale nei valori immobiliari, probabilmente legato alla posizione geografica.

Pattern 2: Età delle Case

I prezzi medi **non** seguono una linea di discesa con l'aumentare dell'età. Dopo un calo iniziale nelle case relativamente giovani, i valori si stabilizzano e mostrano un aumento significativo nelle fasce più alte. Questo "ritorno di fiamma" indica che le case molto vecchie possono acquisire valore storico o perché si trovano in posizioni centrali.

Pattern 3: Affollamento e Valore

La relazione tra affollamento (`AveOccup`) e prezzo è **non lineare**:

- Affollamento moderato (1-3 persone/stanza): prezzi più alti
- Affollamento estremo (>4 persone/stanza): prezzi più bassi

Questo suggerisce che un certo livello di densità abitativa è associato a contesti urbani dinamici, ma il sovraffollamento indica condizioni abitative meno favorevoli.

Pattern 4: Distanza dai Poli Economici

La distanza dal polo economico più vicino (SF o LA) è un predittore migliore della latitudine da sola. Il prezzo tende a diminuire all'aumentare della distanza, riflettendo un effetto di "raffreddamento" del mercato immobiliare.

3. Gestione delle Variabili Categoriche

Le variabili categoriche devono essere convertite in formato numerico prima di essere utilizzate nei modelli di machine learning.

3.1 Tipi di Variabili Categoriche

1. **Binarie**: due sole categorie (Sì/No, Vero/Falso)
2. **Nominali**: categorie senza ordine intrinseco (Città, Colore)
3. **Ordinali**: categorie con ordine naturale (Basso/Medio/Alto)

3.2 Label Encoding

Il **Label Encoding** trasforma le categorie in numeri progressivi:

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
y_encoded = le.fit_transform(['Basso', 'Medio', 'Alto', 'Basso', 'Alto'])
# Output: [1, 2, 0, 1, 0]
```

Quando usare:

- Variabili binarie (Sì/No → 0/1)
- Variabili ordinali (l'ordine è significativo)
- Target di classificazione

Attenzione: non usare Label Encoding per variabili nominali, perché assegna un ordine arbitrario che il modello potrebbe interpretare erroneamente (es. Milano + Roma = Napoli).

3.3 One-Hot Encoding

Il **One-Hot Encoding** crea una colonna binaria (0/1) per ogni categoria:

```
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(sparse_output=False)
ohe_transformed = ohe.fit_transform(df_test[['Città']])
df_ohe = pd.DataFrame(ohe_transformed,
                      columns=ohe.get_feature_names_out(['Città']))
```

Output:

```
   Città_Milano  Città_Napoli  Città_Roma
0           0.0           0.0         1.0
1           1.0           0.0         0.0
2           0.0           1.0         0.0
```

Quando usare:

- Variabili nominali senza ordine intrinseco
- Quando non vuoi che il modello interpreti relazioni numeriche tra categorie

Svantaggio: se una variabile ha molte categorie (es. 100 città), si creano 100 nuove colonne, aumentando la dimensionalità.

Soluzione per molte categorie:

- Usare Target Encoding
- Raggruppare le categorie meno frequenti
- Usare Embedding (per reti neurali)

4. Normalizzazione e Standardizzazione

La **normalizzazione** è necessaria quando le feature hanno scale diverse. Senza normalizzazione, le feature con valori più grandi dominano il modello.

4.1 Il Problema delle Scale Diverse

Considera di minimizzare un errore con due feature:

- Errori di 2 unità sul numero di stanze (media 5.5)
- Errori di 1000 euro sul prezzo di una casa (2 milioni)

Matematicamente, 1000 > 2, ma relativamente parlando, 2 su 5.5 (36%) è un errore più

significativo di 1000 su 2,000,000 (0.05%). La normalizzazione evita questa distorsione.

4.2 MinMaxScaler (Normalizzazione)

Trasforma i dati in un intervallo fisso, solitamente **[0, 1]**:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Pro:

- Mantiene esattamente la relazione tra i dati
- Intervallo prevedibile e fisso

Contro:

- **Estremamente sensibile agli outlier**: un singolo valore estremo schiaccia tutti gli altri valori vicino a 0

```
from sklearn.preprocessing import MinMaxScaler

min_max = MinMaxScaler()
X_minmax = min_max.fit_transform(X)
```

4.3 StandardScaler (Standardizzazione)

Trasforma i dati in modo che abbiano **Media = 0** e **Deviazione Standard = 1**:

$$z = \frac{x - \mu}{\sigma}$$

Pro:

- Più robusto in presenza di outlier
- I coefficienti dei modelli lineari sono più interpretabili
- Requisito per molti algoritmi (regressione logistica, SVM, reti neurali)

Contro:

- Non garantisce un range fisso (i valori possono andare oltre -3 o +3)

```
from sklearn.preprocessing import StandardScaler

standard = StandardScaler()
X_standard = standard.fit_transform(X)
```

4.4 Quando Usare Ogni Scalatore

Scenario	Scalatore Consigliato
Dati con outlier	StandardScaler
Dati senza outlier, bisogno di range fisso	MinMaxScaler
Reti neurali	StandardScaler
Algoritmi basati su distanza (KNN, K-Means)	StandardScaler
Immagini (pixel 0-255)	MinMaxScaler
Modelli lineari	StandardScaler

Regola generale: tranne rare eccezioni, usa **StandardScaler**.

5. Best Practices e Errori Comuni

5.1 Best Practices

1. **Sempre iniziare con EDA**: non saltare mai l'analisi esplorativa iniziale
2. **Controllare sempre min/max dopo `describe()`** : valori fuori scala indicano problemi
3. **Usare sia Pearson che Spearman**: Pearson per relazioni lineari, Spearman per relazioni non lineari
4. **Preferire StandardScaler a MinMaxScaler**: a meno che non ci siano ragioni specifiche per usare un range fisso
5. **Usare One-Hot Encoding per variabili nominali**: mai usare Label Encoding per categorie senza ordine
6. **Creare feature basate su conoscenza del dominio**: feature ingegnerizzate spesso superano feature originali
7. **Verificare la plausibilità dei dati**: non accettare ciecamente i valori del dataset
8. **Documentare le trasformazioni**: tenere traccia di tutte le modifiche applicate ai dati

5.2 Errori Comuni da Evitare

1. **Correlazione = Causalità**: due variabili correlate non significano che una causi l'altra
2. **Ignorare gli outlier**: possono distorcere significativamente l'analisi e le prestazioni del modello
3. **Usare la media globale per imputazione**: quando possibile, usa imputazione condizionata a

variabili correlate
4. **Applicare Z-Score a dati non normali**: Z-Score assume distribuzione normale, usa IQR per dati sconosciuti
5. **Usare Label Encoding per variabili nominali**: introduce un ordine arbitrario che il modello può interpretare erroneamente
6. **Non gestire le scale diverse**: feature con scale diverse dominano il modello
7. **Ignorare la censura dei dati**: alcuni dataset hanno valori massimi/minimi troncati (es. HouseAge max 52)

5.3 Takeaways per la Revisione

1. **EDA è fondamentale**: `describe()` e `info()` sono i primi strumenti da usare su qualsiasi dataset
2. **Outlier detection**: usa Boxplot/IQR per distribuzioni sconosciute, Z-Score per distribuzioni normali
3. **Feature engineering**: crea feature combinando variabili esistenti o basandoti su conoscenza del dominio
4. **Correlazione**: Pearson per lineare, Spearman per non lineare. Ricorda: correlazione ≠ causalità
5. **Normalizzazione**: StandardScaler è la scelta predefinita, MinMaxScaler solo quando serve range fisso
6. **Encoding**: Label Encoding per binarie/ordinali, One-Hot Encoding per nominali
7. **Interpretazione**: i dati spesso rivelano pattern controintuitivi (es. case vecchie che valgono di più per posizione)

6. Riepilogo della Pipeline Completa

Ecco il flusso di lavoro completo per l'analisi di un dataset:

```
1. Caricamento dati
   ↓
2. Info() → verificare struttura, tipi, null
   ↓
3. Describe() → verificare statistiche, min/max, skew
   ↓
4. Boxplot → identificare outlier visivamente
   ↓
5. Rimozione/Trattamento outlier (IQR o Z-Score)
   ↓
6. Gestione missing values (rimuovi, imputa, o predici)
   ↓
7. Pairplot → esplorare relazioni tra variabili
   ↓
8. Matrice di correlazione (Pearson e Spearman)
   ↓
9. Feature engineering (binning, combinazioni, feature derivate)
   ↓
10. Encoding variabili categoriche (One-Hot o Label)
   ↓
11. Normalizzazione/Standardizzazione
   ↓
12. Pronto per il modello ML
```

Questo flusso rappresenta una best practice per la preparazione dei dati prima di qualsiasi compito di machine learning.